IIT HYDERABAD

Department of Electrical Engineering

Digital Systems Laboratory Report

Assignment No: **04**

Course Name: Digital Systems Lab

Course Code: EE1501

Submitted by

Student 1 Name	: Dhanush Sagar
Student 1 Roll No	: EE25BTECH11021
Student 2 Name	: Sreetej Darisy
Student 2 Roll No	: EE25BTECH11018
Branch	: Electrical Engineering
Semester	: II

Submitted to

Gajendranath Chaudhury

Contents

1 Objective **2**

2 Design Methodology **2**

2.1 Clock Design 2

2.2 Sequential Counter Design 2

2.3 Binary to Decimal 3

2.4 Seven Segment Display Design 3

2.5 Display Multiplexing 3

2.6 Output Representation 3

3 Code Implementation **4**

4 Simulation and Verification **8**

5 Hardware Implementation **9**

6 Results and Observations **12**

7 Conclusion **13**

1 Objective

To design and implement a **2-digit digital stopwatch** using the Real Digital Boolean Board (FPGA board) that counts time in seconds from 00 to 59. The stopwatch should display the count in binary form using onboard LEDs and in decimal format using the LCD display. The system must support Start/Stop functionality to control counting, a Reset feature to return the count to 00, and ensure proper rollover from 59 back to 00.

2 Design Methodology

2.1 Clock Design

The input clock signal from the FGPA Board operates at a high frequency (100MHz) and cannot be used directly for second-wise counting. Hence, a clock divider is implemented to generate a slower clock signal.

- It counts up to 50,000,000 cycles. (But , we ran the simulation for only 4 cycles inorder to observe clear and noticeable transition)
- A register (clk_div) is used to divide the incoming clock, and a toggling signal (slow_clk) is generated when the counter reaches a predefined value. This effectively reduces the clock frequency.
- On every rising edge of the input clock, the divider increments.
- When the divider reaches a threshold value, it resets and toggles the slow clock. This slow clock drives the stopwatch counter.
- This approach ensures controlled timing for incrementing the count value.

2.2 Sequential Counter Design

A 6-bit synchronous counter is used to represent values from 0 to 59.

- The counter increments on the rising edge of the generated slow clock. When the run signal is active, counting proceeds otherwise, the counter holds its value.
- When the reset signal is asserted, the counter is cleared to 0.
- If the count reaches 59, it goes back to 0.
- This logic ensures correct stopwatch behavior within the required range.

2.3 Binary to Decimal

Since the counter produces a binary output, it must be converted into decimal digits for display.

- The 6-bit binary value is divided into:
  Tens digit using division by 10 ,
  Ones digit using modulo operation
- These digits are then processed separately for display.

2.4 Seven Segment Display Design

A combinational logic module (bin_to_7seg) is used to convert each decimal digit (0–9) into the corresponding 7-segment display pattern.

- Each input digit maps to a predefined segment configuration.
- Active-low encoding is used for segment control.

Two instances of this module are used:

- One for the tens digit
- One for the ones digit

2.5 Display Multiplexing

To display two digits using limited hardware resources, time-multiplexing is employed.

- A refresh counter is used to alternate between the two digits at high speed.
- Depending on the refresh signal:
- One digit is enabled while the other is disabled
- Corresponding segment data is displayed

This creates the illusion of simultaneous display of both digits.

2.6 Output Representation

- The binary count is directly connected to LEDs for visualization.
- The decimal output is shown on the 7-segment display using multiplexing.

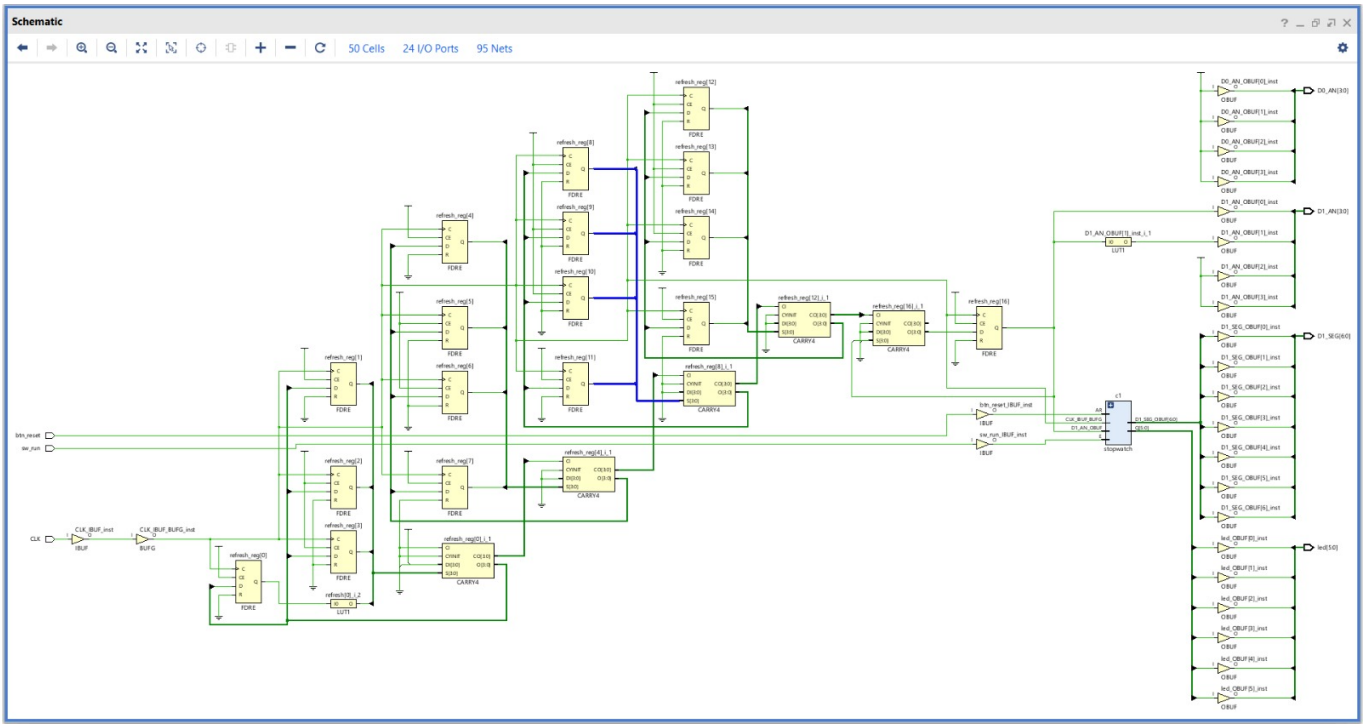

Figure 1: Schematic

3 Code Implementation

Listing 1: Design Code

```
1
2  `timescale 1ns / 1ps
3
4  module Boolean_Board_Top (
5      input CLK, sw_run, btn_reset,
6      output [5:0] led,
7      output [6:0] D1_SEG,
8      output [3:0] D1_AN,
9      output [3:0] D0_AN
10 );
11     wire [5:0] count;
12     wire [6:0] tens, ones;
13     reg [16:0] refresh = 0;
14
15     // 1. Logic Instances
16     stopwatch c1 (.sClock(CLK), .reset(btn_reset), .run(sw_run),
           .sCount(count));
17     dual_digit_controller d1 (.binary_in({1'b0, count}), .
           seg_tens(tens), .seg_ones(ones));
```

```
18
19      // 2. Refresh Counter (The only "extra" part)
20      always @(posedge CLK) refresh <= refresh + 1;
21
22      assign D1_AN  = refresh[16] ? 4'b1101 : 4'b1110;
23      assign D1_SEG = refresh[16] ? tens    : ones;
24
25      // 4. Turn off the other display and set LEDs
26      assign D0_AN = 4'b1111;
27      assign led = count;
28
29  endmodule
30
31  module bin_to_7seg (
32      input [3:0] bin,       // 4-bit binary input (0-9)
33      output reg [6:0] seg   // 7-bit segment output (a-g)
34  );
35
36   always @(*) begin
37          case (bin)
38              4'b0000: seg = ~7'b0111111; // 0
39              4'b0001: seg = ~7'b0000110; // 1
40              4'b0010: seg = ~7'b1011011; // 2
41              4'b0011: seg = ~7'b1001111; // 3
42              4'b0100: seg = ~7'b1100110; // 4
43              4'b0101: seg = ~7'b1101101; // 5
44              4'b0110: seg = ~7'b1111101; // 6
45              4'b0111: seg = ~7'b0000111; // 7
46              4'b1000: seg = ~7'b1111111; // 8
47              4'b1001: seg = ~7'b1101111; // 9
48              default: seg = ~7'b0000000;
49          endcase
50      end
51  endmodule
52
53  module dual_digit_controller (
54      input [6:0] binary_in,     // 7-bit input (covers 0 to 99)
55      output [6:0] seg_tens,     // 7-bit output for the Tens digit
56      output [6:0] seg_ones      // 7-bit output for the Ones digit
57  );
58      wire [3:0] tens_val;
```

```
59      wire [3:0] ones_val;
60
61      // Split the number into two digits
62      assign tens_val = binary_in / 10;
63      assign ones_val = binary_in % 10;
64
65      // Instance 1: Decoder for the Tens digit
66      bin_to_7seg tens_display (
67          .bin(tens_val),
68          .seg(seg_tens)
69      );
70
71      // Instance 2: Decoder for the Ones digit
72      bin_to_7seg ones_display (
73          .bin(ones_val),
74          .seg(seg_ones)
75      );
76
77  endmodule
78
79  module stopwatch(
80      input sClock,
81      input reset,
82      input run,
83      output reg [5:0] sCount = 6'b000000
84      );
85
86      reg [26:0] clk_div = 0;
87      reg slow_clk = 0;
88
89      always @(posedge sClock) begin
90          if (clk_div == 4) begin
91              clk_div <= 0;
92              slow_clk <= ~slow_clk;
93          end else begin
94              clk_div <= clk_div + 1;
95          end
96      end
97
98      always @(posedge slow_clk or posedge reset) begin
99          if (reset) begin
```

```
            sCount <= 6'd0;
        end
        else if (run) begin // Only count if Start/Stop is HIGH
            if (sCount >= 59)
                sCount <= 0;
            else
                sCount <= sCount + 1;
        end
    end
endmodule
```

Listing 2: Testbench Code

```

`timescale 1ns / 1ps

module stopwatch_tb();
    // 1. Signals for the Counter
    reg sClock;
    reg reset;
    reg run;
    wire [5:0] binary_count; // This wire holds the binary value
       (0-59)

    // 2. Signals for the LED Mapping
    wire [6:0] tens_segments;
    wire [6:0] ones_segments;

    stopwatch uut_counter (
        .sClock(sClock),
        .reset(reset),
        .run(run),
        .sCount(binary_count)
    );

    dual_digit_controller uut_display (
        .binary_in({1'b0, binary_count}),
        .seg_tens(tens_segments),
        .seg_ones(ones_segments)
    );

    initial sClock = 0;
```

```
29      always #5 sClock = ~sClock;
30
31      initial begin
32          // Reset the system
33          reset = 1; run = 0; #20;
34          reset = 0; #20;
35
36          // Start counting
37          run = 1;
38          #3000;
39
40          $finish;
41      end
42  endmodule
```

4 Simulation and Verification

Figure 2: Simulation

- The testbench simulation was carried out to validate the functional correctness of the digital stopwatch design in a controlled environment. A periodic clock signal with a 10 ns time period was generated using an always block to drive the system.

- Initially, the reset signal was asserted while keeping the run signal low to initialize the counter to zero. After a brief delay, the reset was deasserted, and the run signal was set high to enable counting.
- The simulation was executed for a sufficient duration to observe multiple count transitions.
- The binary_count output was monitored to verify correct sequential incrementing from 0 onwards, while the corresponding tens_segments and ones_segments outputs were analyzed to confirm accurate binary-to-decimal conversion and proper 7-segment encoding.
- The results demonstrate that the stopwatch operates correctly under different control conditions, validating both counting and display logic.

Test Case	Input Condition	Expected Output
1	$reset = 1,\ run = 0$	Counter resets to 0
2	$reset = 0,\ run = 0$	Counter holds previous value
3	$reset = 0,\ run = 1$	Counter increments sequentially
4	Count reaches 59	Counter rolls over to 0
5	Binary input changes	Correct update in 7-segment outputs

5 Hardware Implementation

Constraints :

```
create_clock -period 10.000 -name sys_clk_pin -waveform {0.000 5.000} [get_ports CLK]
set_property PACKAGE_PIN K1 [get_ports sw_run]
set_property PACKAGE_PIN F14 [get_ports CLK]
set_property PACKAGE_PIN J2 [get_ports btn_reset]
set_property PACKAGE_PIN G1 [get_ports {LED[0]}]
set_property PACKAGE_PIN G2 [get_ports {LED[1]}]
set_property PACKAGE_PIN F1 [get_ports {LED[2]}]
set_property PACKAGE_PIN F2 [get_ports {LED[3]}]
set_property PACKAGE_PIN E1 [get_ports {LED[4]}]
set_property PACKAGE_PIN E2 [get_ports {LED[5]}]
set_property PACKAGE_PIN F4 [get_ports {D1_SEG[0]}]
set_property PACKAGE_PIN J3 [get_ports {D1_SEG[1]}]
set_property PACKAGE_PIN D2 [get_ports {D1_SEG[2]}]
set_property PACKAGE_PIN C2 [get_ports {D1_SEG[3]}]
set_property PACKAGE_PIN B1 [get_ports {D1_SEG[4]}]
```

```
set_property PACKAGE_PIN H4 [get_ports {D1_SEG[5]}]
set_property PACKAGE_PIN D1 [get_ports {D1_SEG[6]}]
set_property PACKAGE_PIN H3 [get_ports {D1_AN[0]}]
set_property PACKAGE_PIN J4 [get_ports {D1_AN[1]}]
set_property PACKAGE_PIN F3 [get_ports {D1_AN[2]}]
set_property PACKAGE_PIN E4 [get_ports {D1_AN[3]}]
set_property PACKAGE_PIN D7 [get_ports {D0_SEG[0]}]
set_property PACKAGE_PIN C5 [get_ports {D0_SEG[1]}]
set_property PACKAGE_PIN A5 [get_ports {D0_SEG[2]}]
set_property PACKAGE_PIN B7 [get_ports {D0_SEG[3]}]
set_property PACKAGE_PIN A7 [get_ports {D0_SEG[4]}]
set_property PACKAGE_PIN D6 [get_ports {D0_SEG[5]}]
set_property PACKAGE_PIN B5 [get_ports {D0_SEG[6]}]
set_property PACKAGE_PIN D5 [get_ports {D0_AN[0]}]
set_property PACKAGE_PIN C4 [get_ports {D0_AN[1]}]
set_property PACKAGE_PIN C7 [get_ports {D0_AN[2]}]
set_property PACKAGE_PIN A8 [get_ports {D0_AN[3]}]

set_property IOSTANDARD LVCMOS33 [get_ports {D0_AN[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {D0_AN[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {D0_AN[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {D0_AN[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {D0_SEG[6]}]
set_property IOSTANDARD LVCMOS33 [get_ports {D0_SEG[5]}]
set_property IOSTANDARD LVCMOS33 [get_ports {D0_SEG[4]}]
set_property IOSTANDARD LVCMOS33 [get_ports {D0_SEG[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {D0_SEG[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {D0_SEG[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {D0_SEG[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {D1_AN[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {D1_AN[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {D1_AN[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {D1_AN[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {D1_SEG[6]}]
set_property IOSTANDARD LVCMOS33 [get_ports {D1_SEG[5]}]
set_property IOSTANDARD LVCMOS33 [get_ports {D1_SEG[4]}]
set_property IOSTANDARD LVCMOS33 [get_ports {D1_SEG[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {D1_SEG[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {D1_SEG[1]}]
```

```
set_property IOSTANDARD LVCMOS33 [get_ports {D1_SEG[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {LED[5]}]
set_property IOSTANDARD LVCMOS33 [get_ports {LED[4]}]
set_property IOSTANDARD LVCMOS33 [get_ports {LED[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {LED[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {LED[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {LED[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports btn_reset]
set_property IOSTANDARD LVCMOS33 [get_ports CLK]
set_property IOSTANDARD LVCMOS33 [get_ports sw_run]

set_property IOSTANDARD LVCMOS33 [get_ports {led[5]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led[4]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led[0]}]
set_property PACKAGE_PIN E2 [get_ports {led[5]}]
set_property PACKAGE_PIN E1 [get_ports {led[4]}]
set_property PACKAGE_PIN F2 [get_ports {led[3]}]
set_property PACKAGE_PIN F1 [get_ports {led[2]}]
set_property PACKAGE_PIN G2 [get_ports {led[1]}]
set_property PACKAGE_PIN G1 [get_ports {led[0]}]
```

The design was mapped to the Spartan-7 FPGA on the Boolean Board using the following physical pin constraints :

Signal Name	**Port**	**FPGA Pin**
Clock	CLK	F14
Run Control	sw_run	K1
Reset Button	btn_reset	J2
LED0	$led[0]$	G1
LED1	$led[1]$	G2
LED2	$led[2]$	F1
LED3	$led[3]$	F2
LED4	$led[4]$	E1
LED5	$led[5]$	E2
7-Segment (D1) a	$D1\_SEG[0]$	F4

Signal Name	Port	FPGA Pin
7-Segment (D1) b	$D1\_SEG[1]$	J3
7-Segment (D1) c	$D1\_SEG[2]$	D2
7-Segment (D1) d	$D1\_SEG[3]$	C2
7-Segment (D1) e	$D1\_SEG[4]$	B1
7-Segment (D1) f	$D1\_SEG[5]$	H4
7-Segment (D1) g	$D1\_SEG[6]$	D1
Digit Select (D1)	$D1\_AN[0]$	H3
Digit Select (D1)	$D1\_AN[1]$	J4
Digit Select (D1)	$D1\_AN[2]$	F3
Digit Select (D1)	$D1\_AN[3]$	E4
7-Segment (D0) a	$D0\_SEG[0]$	D7
7-Segment (D0) b	$D0\_SEG[1]$	C5
7-Segment (D0) c	$D0\_SEG[2]$	A5
7-Segment (D0) d	$D0\_SEG[3]$	B7
7-Segment (D0) e	$D0\_SEG[4]$	A7
7-Segment (D0) f	$D0\_SEG[5]$	D6
7-Segment (D0) g	$D0\_SEG[6]$	B5
Digit Select (D0)	$D0\_AN[0]$	D5
Digit Select (D0)	$D0\_AN[1]$	C4
Digit Select (D0)	$D0\_AN[2]$	C7
Digit Select (D0)	$D0\_AN[3]$	A8

6 Results and Observations

The working demonstration of the circuit is available at DEMO.

- The counter increments sequentially only when the run signal is active.
- When the reset signal is asserted, the counter immediately returns to 00.
- The stopwatch correctly rolls over from 59 to 00, confirming proper limit handling.
- The LEDs display the binary count in real time, matching the decimal value shown on the 7-segment display.
- The multiplexing of the 7-segment display is fast enough to appear as a steady two-digit output to the human eye.
- The binary-to-decimal conversion using division and modulo operations produces accurate digit separation.

- Simulation results match hardware behavior, confirming functional correctness of the design

7 Conclusion

The 2-digit digital stopwatch was successfully implemented on the Real Digital Boolean Board using Verilog. It correctly counts from 00 to 59 and rolls over to 00. The binary output is shown on LEDs, and the decimal output is displayed on a multiplexed 7-segment display. The Start/Stop and Reset functions work as expected. Simulation and hardware results match, confirming correct operation. The project demonstrates practical use of counters, clock division, and display interfacing.